

Projet Linked Crossing - Rendu 2

Contributeurs: 399554, 397957

Date: 27 avril 2025

• Hiérarchie de Classes des Entités Mobiles

Notre système d'entités mobiles utilise l'héritage pour maximiser la réutilisation du code. La classe de base **Mobile** définit les attributs communs (position, angle d'orientation, déplacement) et déclare des fonctions virtuelles que les classes dérivées doivent implémenter (**move()**, **isValid()**, **isInArena()**, **draw()**).

Particule étend **Mobile** avec un compteur pour la mécanique de division. Lorsque le compteur atteint **time_to_split**, elle se divise en deux nouvelles particules avec des trajectoires modifiées ou est détruite si le nombre maximal de particules est atteint.

Faiseur étend **Mobile** avec un rayon, un nombre d'éléments et les positions des éléments. Il implémente une détection de collision spécialisée et une logique de mouvement, assurant que ses éléments suivent le chemin historique de sa tête tout en respectant les limites de l'arène et en évitant les collisions avec d'autres faiseurs.

Cette hiérarchie permet une gestion polymorphe des différents types d'entités à travers une collection unique, simplifiant la gestion des entités tout en préservant les comportements spécifiques à chaque type.

• Structure de Données de la Classe Jeu

La classe **Jeu** gère l'état du jeu avec ces composants clés:

- **Stockage des entités:** Un vecteur de **unique_ptr<Mobile>** contient toutes les entités, avec des vecteurs d'indices séparés pour les particules et les faiseurs permettant un accès typé sécurisé.
- **État du jeu:** Des variables suivent le score, l'état du jeu et le chemin du fichier pour la fonctionnalité de redémarrage.
- **Gestion de la chaîne:** Un objet **Chaine** gère l'état de la chaîne articulée.

La classe implémente les opérations de fichier via une machine à états et gère les mises à jour du jeu à travers des méthodes distinctes pour les particules et les faiseurs. La logique de mise à jour suit la séquence spécifiée dans les directives du projet: décrémenter le score, mettre à jour les particules (y compris division/destruction), mettre à jour les faiseurs (avec détection de collision), et gérer les opérations liées à la chaîne.

Cette conception sépare proprement la logique du jeu des préoccupations de l'interface graphique tout en fournissant une interface simple pour accéder et modifier l'état du jeu.

● Implémentation du Module Tools

Le module **tools** fournit des utilitaires géométriques indépendants de la logique spécifique au jeu:

- **S2d**: Structure simple de coordonnées 2D utilisée dans tout le projet
- **Point**: Classe de base pour la gestion des positions
- **Circle**: Étend Point avec des capacités de rayon et de test d'intersection
- **Vector**: Gère la direction, la magnitude et des opérations comme les calculs de rebond

Des fonctions utilitaires complètent ces classes avec des calculs de distance, détection de collision et opérations de dessin. Le module maintient une stricte indépendance des concepts du jeu, suivant le principe de séparation des préoccupations et permettant une réutilisation potentielle.

● Activité Individuelle

Pour le Rendu 2, nous nous sommes concentrés sur nos domaines d'expertise respectifs tout en assurant une intégration complète:

Je (399554) me suis concentré sur les composants du Modèle:

- Implémentation de la classe **Jeu** avec gestion des fichiers et de l'état du jeu
- Développement de la hiérarchie des entités mobiles avec conception polymorphe
- Création de la logique de mise à jour du jeu suivant les règles spécifiées

Karol (397957) s'est concentré sur l'interface graphique et les outils:

- Implémentation du module **tools** avec des utilitaires géométriques
- Développement du système de rendu graphique avec mise à l'échelle appropriée
- Création des composants d'interface graphique pour l'interaction utilisateur

Inspirés par certaines publications sur LinkedIn, nous essayons d'introduire des "soirées démo" où nous "pitch" notre code l'un à l'autre, assurant une compréhension partagée malgré des domaines de spécialisation différents. Pour les points d'intégration complexes, nous avons utilisé la programmation en binôme pour assurer une interaction cohérente entre les sous-systèmes.